Shortest Path Optimization on University Campus Using Dijkstra's Algorithm

Aditya Vikram Meda, J. Ashmita, Sahasri Gundapuneedi, Zoya Bothra

November 2024

1 Introduction

Dijkstra's algorithm is used to find the shortest path from the source vertex to all the other vertices in a graph. The graph can be either directed or undirected and the edges must have non-negative weights.

Steps :

1. Set initial distances for all vertices: 0 for the source vertex, and infinity for all the other vertices.
2. Choose the unvisited vertex with the smallest known distance from the source as the current vertex. Initially, this will be the source vertex.
3. For each unvisited neighbor of the current vertex, calculate the tentative distance (current node's distance + edge weight) from the source. If this new distance is smaller than the currently known distance, update the neighbor's distance.
4. Once all neighbors of the current vertex have been processed, mark the current vertex as visited. Visited vertices will not be checked again.
5. Move to the unvisited vertex with the smallest tentative distance and repeat steps 3 and 4 until all vertices have been visited.
6. In the end we are left with the shortest path from the source vertex to every other vertex in the graph.

The distance from the source to any vertex v is determined by

$$dist[v] = min(dist[v], (dist[u] + w(u, v))) \tag{1}$$

Where,

- dist[v] : The current known shortest distance from the source to vertex v.
- dist[u] : The current known shortest distance from the source to vertex u.
- w(u,v) : The weight of the edge from vertex u to vertex v.

2 Data collection methods

The data was collected by mapping key locations across the university campus. Each location was assigned a unique node to simplify representation in the adjacency matrix. Distances between locations were measured by walking around the campus and checking with Google Maps for accuracy. For areas not available on Google Maps, we measured the distances manually. These distances, in meters, were used as weights for the paths, and an adjacency matrix was created to show the connections between locations. The final map we have obtained is :

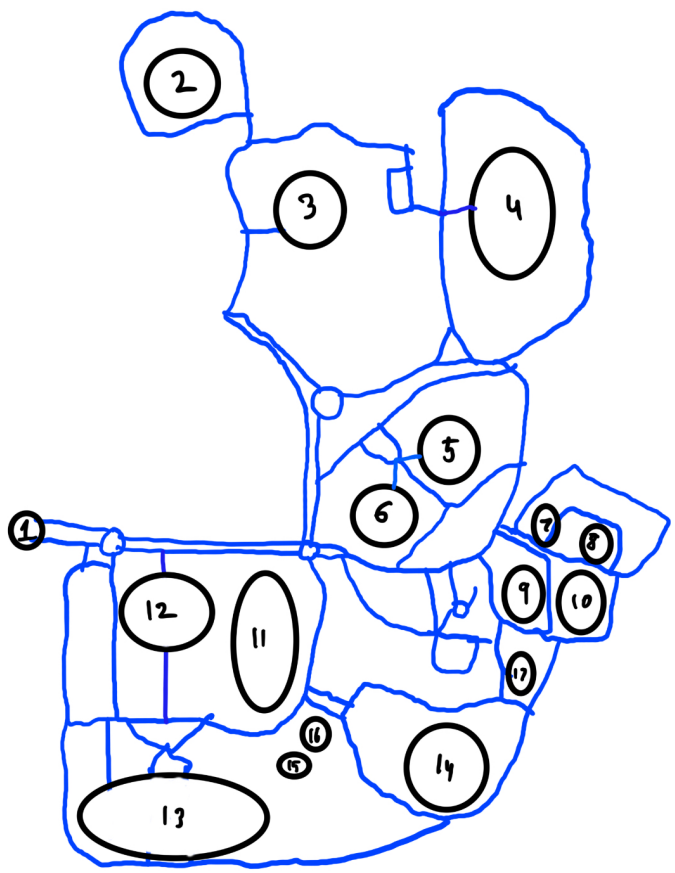

Figure 1: Map of the campus

Where the vertices of the map are as follows:

No.	**Location**
1	Main Gate
2	Cricket Ground
3	Sports Area
4	Library
5	IT Block-1
6	IT Block-2
7	Phase 2 Hostel
8	Cafeteria
9	Phase 3 Hostel
10	Phase 1 Hostel
11	Football Ground
12	BBA / Law Block
13	Girls Hostel
14	Engineering Block
15	Badminton Court
16	Tennis Court
17	Mechanical Labs

Table 1: List of Locations

And the adjacency matrix of the weighted graph is :

1	0	750	650	700	400	375	500	510	500	510	350	125	350	450	475	450	600
2	750	0	100	300	650	650	750	760	750	760	600	625	400	700	705	700	800
3	650	100	0	400	600	610	550	560	550	560	450	500	550	510	515	510	650
4	700	300	400	0	350	360	250	260	250	260	375	425	450	425	430	425	650
5	400	650	600	350	0	10	150	155	150	155	200	225	300	225	230	225	300
6	375	650	610	360	10	0	150	155	150	155	200	225	300	225	230	225	300
7	500	750	550	250	150	150	0	5	5	5	225	310	350	250	255	250	100
8	510	760	560	260	155	155	5	0	5	5	225	310	350	250	225	250	100
9	500	750	550	250	150	150	5	5	0	5	225	310	350	250	225	250	100
10	510	760	560	260	155	155	5	5	5	0	225	310	350	250	225	250	100
11	350	600	450	375	200	200	225	225	225	225	0	175	125	50	50	50	150
12	125	625	500	425	225	225	310	310	310	310	175	0	100	150	100	150	275
13	350	400	550	450	300	300	350	350	350	350	125	100	0	125	100	125	225
14	450	700	510	425	225	225	250	250	250	250	50	150	125	0	15	15	100
15	475	705	515	430	230	230	255	225	225	225	50	100	100	15	0	15	125
16	450	700	510	425	225	225	250	250	250	250	50	150	125	15	15	0	105
17	600	800	650	650	300	300	100	100	100	100	150	275	225	100	125	105	0

Table 2: Distance Matrix

3 Implementation

Here is the implementation of Dijkstra's Algorithm in C for the given adjacency matrix:

```c
#include <stdio.h>
#include <limits.h>

#define V 17 // Number of vertices

// Function to find the vertex with the minimum distance value
int minDistance(int dist[], int sptSet[]) {
    int min = INT_MAX, min_index;

    for (int v = 0; v < V; v++) {
        if (sptSet[v] == 0 && dist[v] <= min) {
            min = dist[v];
            min_index = v;
        }
    }
    return min_index;
}

// Function to print the shortest path from source to a target
void printPath(int parent[], int target) {
    if (parent[target] == -1) {
        printf("%d ", target + 1); // Convert to 1-indexed for
            readability
        return;
    }
    printPath(parent, parent[target]);
    printf("%d ", target + 1);
}

// Function to implement Dijkstra's algorithm
void dijkstra(int graph[V][V], int src, int target) {
    int dist[V];       // Array to store the shortest distance from
         the source to each vertex
    int sptSet[V];     // Shortest path tree set (true if vertex is
         included in the shortest path)
    int parent[V];     // Array to store the shortest path tree

    // Initialize distances as infinite and sptSet[] as false
    for (int i = 0; i < V; i++) {
        dist[i] = INT_MAX;
        sptSet[i] = 0;
        parent[i] = -1; // No parent initially
    }

    // Distance from source to itself is always 0
    dist[src] = 0;

    // Find the shortest path for all vertices
    for (int count = 0; count < V - 1; count++) {
        // Pick the minimum distance vertex from the set of
            vertices not yet processed
```

```
        int u = minDistance(dist, sptSet);

        // Mark the picked vertex as processed
        sptSet[u] = 1;

        // Update the distance value of the adjacent vertices of
            the picked vertex
        for (int v = 0; v < V; v++) {
            // Update dist[v] if not in sptSet, there is an edge,
                and total distance is smaller
            if (!sptSet[v] && graph[u][v] && dist[u] != INT_MAX
                && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
                parent[v] = u; // Record the path
            }
        }
    }

    // Output the shortest path and distance
    printf("Shortest distance from %d to %d: %d\n", src + 1, target
         + 1, dist[target]);
    printf("Path: ");
    printPath(parent, target);
    printf("\n");
}

int main() {
    int graph[V][V] = {
        {0, 750, 650, 700, 400, 375, 500, 510, 500, 510, 350, 125,
            350, 450, 475, 450, 600},
        {750, 0, 100, 300, 650, 650, 750, 760, 750, 760, 600, 625,
            400, 700, 705, 700, 800},
        {650, 100, 0, 400, 600, 610, 550, 560, 550, 560, 450, 500,
            550, 510, 515, 510, 650},
        {700, 300, 400, 0, 350, 360, 250, 260, 250, 260, 375, 425,
            450, 425, 430, 425, 650},
        {400, 650, 600, 350, 0, 10, 150, 155, 150, 155, 200, 225,
            300, 225, 230, 225, 300},
        {375, 650, 610, 360, 10, 0, 150, 155, 150, 155, 200, 225,
            300, 225, 230, 225, 300},
        {500, 750, 550, 250, 150, 150, 0, 5, 5, 5, 225, 310, 350,
            250, 255, 250, 100},
        {510, 760, 560, 260, 155, 155, 5, 0, 5, 5, 225, 310, 350,
            250, 225, 250, 100},
        {500, 750, 550, 250, 150, 150, 5, 5, 0, 5, 225, 310, 350,
            250, 225, 250, 100},
        {510, 760, 560, 260, 155, 155, 5, 5, 5, 0, 225, 310, 350,
            250, 225, 250, 100},
        {350, 600, 450, 375, 200, 200, 225, 225, 225, 225, 0, 175,
            125, 50, 50, 50, 150},
        {125, 625, 500, 425, 225, 225, 310, 310, 310, 310, 175, 0,
            100, 150, 100, 150, 275},
        {350, 400, 550, 450, 300, 300, 350, 350, 350, 350, 125,
            100, 0, 125, 100, 125, 225},
        {450, 700, 510, 425, 225, 225, 250, 250, 250, 250, 50, 150,
             125, 0, 15, 15, 100},
        {475, 705, 515, 430, 230, 230, 255, 225, 225, 225, 50, 100,
```

```
            100, 15, 0, 15, 125},
        {450, 700, 510, 425, 225, 225, 250, 250, 250, 250, 50, 150,
            125, 15, 15, 0, 105},
        {600, 800, 650, 650, 300, 300, 100, 100, 100, 100, 150,
            275, 225, 100, 125, 105, 0}
    };

    int source, target;

    // User input for source and target vertices
    printf("Enter the source vertex (1 to 17): ");
    scanf("%d", &source);
    printf("Enter the target vertex (1 to 17): ");
    scanf("%d", &target);

    // Validate input
    if (source < 1 || source > V || target < 1 || target > V) {
        printf("Invalid vertex.\n");
        return 1;
    }

    // Call Dijkstra's algorithm
    dijkstra(graph, source - 1, target - 1); // Adjust 1-indexed
        input to 0-indexed

    return 0;
}
```

4 Result and Analysis

Results : The shortest distance from the ECOSE building to the other vertices are :

Source Vertex	Shortest Distance	Path
1	240	$14 \to 15 \to 12 \to 1$
2	515	$14 \to 15 \to 13 \to 2$
3	500	$14 \to 11 \to 3$
4	425	$14 \to 4$
5	225	$14 \to 5$
6	225	$14 \to 6$
7	200	$14 \to 17 \to 7$
8	200	$14 \to 17 \to 8$
9	200	$14 \to 17 \to 9$
10	200	$14 \to 17 \to 10$
11	50	$14 \to 11$
12	115	$14 \to 15 \to 12$
13	115	$14 \to 15 \to 13$
14	0	14
15	15	$14 \to 15$
16	15	$14 \to 16$
17	100	$14 \to 17$

Table 3: Shortest Distances and Paths from Source Vertex 14

Analysis : If any of the edge weights is changed due to the closure of a path, the shortest path to any of the vertices that includes that edge will change. Hence , Dijkstra's algorithm will need to be rerun to ensure that the correct shortest path is selected. This recalculation is necessary because the algorithm relies on the assumption that the previously found shortest paths are still valid, which may no longer be true after an edge weight change.

Edge Weight Decreases If the weight of an edge decreases, it might create a shorter path between two nodes. This change could invalidate the previously calculated shortest path.

- **Effect:** The current shortest path may no longer be optimal because the reduced edge weight could offer a better route.
- **Action:** After the edge weight is updated, Dijkstra's algorithm must be rerun or recalculated, starting from the affected node or the source node, depending on the scope of the change.

For example, if a path that previously had a higher cost becomes cheaper due to the updated edge weight, Dijkstra's algorithm needs to check if this new path is now shorter than the previously calculated shortest path.

Edge Weight Increases If the weight of an edge increases, it might make the current path between two nodes less optimal than before.

- **Effect:** The increase in the edge weight may invalidate the current shortest path, making it suboptimal.
- **Action:** Similarly, the algorithm must be rerun to ensure the shortest path is recalculated, as the path with the higher edge weight may no longer be optimal.